GenomicRanges - Rle

Kasper D. Hansen

- Dependencies
- Corrections
  - Overview
- Other Resources
  - Coverage
  - Rle
  - Useful functions for Rle
  - Views and Rles
  - RleList
  - Rles and GRanges
- Biology Usecase
- SessionInfo

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

In this session we will discuss a data representation class called `Rle` (run length encoding). This class is great for representation genome-wide sequence coverage.

Other Resources

Coverage

In high-throughput sequencing, coverage is the number of reads overlapping each base. In other words, it associates a number (the number of reads) to every base in the genome.

This is a fundamental quantity for many high-throughout sequencing analyses. For variant calling (DNA sequencing) it tells you how much power (information) you have to call a variant at a given location. For ChIP sequencing it is the primary signal; areas with high coverage are thought to be enriched for a given protein.

A file format which is often used to represent coverage data is `Wig` or the modern version `BigWig`.

Rle

An `Rle` (run-length-encoded) vector is a specific representation of a vector. The *IRanges* package implements support for this class. Watch out: there is also a base R class called `rle` which has much less functionality.

The run-length-encoded representation of a vector, represents the vector as a set of distinct runs with their own value. Let us take an example

```
rl <- Rle(c(1,1,1,1,2,2,3,3,2,2))
rl
```

```
## numeric-Rle of length 10 with 4 runs
##   Lengths: 4 2 2 2
##   Values : 1 2 3 2
```

```
runLength(rl)
```

```
## [1] 4 2 2 2
```

```
runValue(rl)
```

```
## [1] 1 2 3 2
```

```
as.numeric(rl)
```

```
##  [1] 1 1 1 1 2 2 3 3 2 2
```

Note the accessor functions `runLength()` and `runValue()`.

This is a very efficient representation if

- the vector is very long
- there are a lot of consecutive elements with the same value

This is especially useful for genomic data which is either piecewise constant, or where most of the genome is not covered (eg. RNA sequencing in mammals).

In many ways `Rle`s function as normal vectors, you can do arithmetic with them, transform them etc. using standard R functions like `+` and `log2`.

There are also `RleList` which is a list of `Rle`s. This class is used to represent a genome wide coverage track where each element of the list is a different chromosome.

Useful functions for Rle

A standard usecase is that you have a number of regions (say `IRanges`) and you want to do something to your `Rle` over each of these regions. Enter `aggregate()`.

```
ir <- IRanges(start = c(2,6), width = 2)
aggregate(rl, ir, FUN = mean)
```

```
## [1] 1.0 2.5
```

It is also possible to covert an `IRanges` to a `Rle` by the `coverage()` function. This counts, for each integer, how many ranges overlap the integer.

```
ir <- IRanges(start = 1:10, width = 3)
rl <- coverage(ir)
rl
```

```
## integer-Rle of length 12 with 5 runs
##   Lengths: 1 1 8 1 1
##   Values : 1 2 3 2 1
```

You can select high coverage regions by the `slice()` function:

```
slice(rl, 2)
```

```
## Views on a 12-length Rle subject
##
## views:
##     start end width
## [1]     2  11    10 [2 3 3 3 3 3 3 3 3 2]
```

This outputs a `Views` object, see next section.

Views and Rles

In the sessions on the ***Biostrings*** package we learned about `Views` on genomes. `Views` can also be instantiated on `Rles`.

```
vi <- Views(rl, start = c(3,7), width = 3)
vi
```

```
## Views on a 12-length Rle subject
##
## views:
##     start end width
## [1]     3   5     3 [3 3 3]
## [2]     7   9     3 [3 3 3]
```

with `Views` you can now (again) apply functions:

```
mean(vi)
```

```
## [1] 3 3
```

This is very similar to using `aggregate()` described above.

RleList

An `RleList` is simply a list of `Rle`. It is similar to a `GRangesList` in concept.

Rles and GRanges

`Rle`'s can also be constructed from `GRanges`.

This often involves `RleList` where each element of the list is a chromosome. Surprisingly, we do not yet have an `RleList` type structure which also contains information about say the length of the different chromosomes.

Let us see some examples

```
gr <- GRanges(seqnames = "chr1", ranges = IRanges(start = 1:10, width =
rl <- coverage(gr)
rl
```

```
## RleList of length 1
## $chr1
## integer-Rle of length 12 with 5 runs
##   Lengths: 1 1 8 1 1
##   Values : 1 2 3 2 1
```

Using `Views` on such an object exposes some missing functionality

```
grView <- GRanges("chr1", ranges = IRanges(start = 2, end = 7))
vi <- Views(rl, grView)
```

```
## Error in RleViewsList(rleList = subject, rangesList = start): 'ranges
```

We get an error, mentioning some object called a `RangesList`. This type of object is similar to a `GRanges` and could be considered succeeded by the later class. We sometimes see instances of this popping around.

```
vi <- Views(rl, as(grView, "RangesList"))
vi
```

```
## RleViewsList of length 1
## names(1): chr1
```

```
vi[[1]]
```

```
## Views on a 12-length Rle subject
##
## views:
##     start end width
## [1]     2   7     6 [2 3 3 3 3 3]
```

Biology Usecase

Suppose we want to compute the average coverage of bases belonging to (known) exons.

Input objects are
reads: a `GRanges`.
exons: a `GRanges`.

pseudocode:

```
bases <- reduce(exons)
nBases <- sum(width(bases))
nCoverage <- sum(Views(coverage(reads), bases))
nCoverage / nBases
```

(watch out for strand)

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
## [1] GenomicRanges_1.20.5 GenomeInfoDb_1.4.2   IRanges_2.2.7
## [4] S4Vectors_0.6.3      BiocGenerics_0.14.0  BiocStyle_1.6.0
## [7] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.8    formatR_1.2     magrittr_1.5    evaluate_0.7.2
##  [5] stringi_0.5-5   XVector_0.8.0   tools_3.2.1     stringr_1.0.0
##  [9] yaml_2.1.13     htmltools_0.2.6 knitr_1.11
```